SPVS final setup and demonstration guide

Use the source ZIP from this completion branch and read FINAL_COMPLETION_REPORT.md before presenting capabilities. Keep the baseline release intact. Commands below are for a MacBook Air M4 with macOS and a trusted development network. The Linux execution report does not claim these Mac commands were run on Apple Silicon.

1. Prerequisites

Install Homebrew from its official instructions, Docker Desktop for Apple Silicon, Android Studio for Apple Silicon, and Xcode from the App Store. Launch each application once, accept its license and complete first-run components. In Xcode Settings > Locations select the active command-line tools. Then install:

```
brew install python@3.12 node@22 uv openjdk@17 ruby
export PATH="$(brew --prefix node@22)/bin:$(brew --prefix ruby)/bin:$PATH"
export JAVA_HOME="$(brew --prefix openjdk@17)/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$ANDROID_HOME/platform-tools:$PATH"
xcodebuild -version
node --version
java -version
```

Use Python 3.12, Node 22 (or the verified Node 24 environment), JDK 17 and the pinned npm/uv lockfiles. React Native's checked-in helper requires at least Xcode 16.1; install a current compatible Xcode and supported iOS simulator runtime. In Android Studio's SDK Manager install the SDK, build tools, CMake and NDK versions reported by the generated project. This run used Android API/build tools 37, Gradle 9.4.1 and NDK 27.1.12297006. Let official tools install missing exact versions. Do not substitute unofficial Maven mirrors or disable TLS.

2. Local services and accounts

From the unpacked `spvs-root` directory:

```
python3 scripts/init_env.py
uv sync --project backend --frozen
python3 scripts/download_models.py
npm ci --prefix dashboard
npm ci --prefix mobile
docker compose up -d db --wait
bash scripts/backend.sh alembic upgrade head
bash scripts/backend.sh alembic check
bash scripts/backend.sh python -m app.seed
```

The setup script creates local `.env` secrets and a random DEMO_PASSWORD. Read that password locally; never paste it into reports or commit the file. Demo accounts are admin@spvs.example, faculty@spvs.example and student@spvs.example. Seed creates no attendance or biometrics. Change the demo classroom to real coordinates and permitted real AP references before phone testing.

Run in separate terminals:

```
make api
make dashboard
```

The API listens on port 8000 and the dashboard on 5173. Alternatively use `docker compose up --build -d` after models are downloaded. Open http://localhost:5173. `/health` checks PostgreSQL and `/ready` also loads

the real models. Use trusted TLS for any real data. Create individual accounts for institutional testing instead of shared demo credentials.

3. Automated verification

Create an isolated test database once, without changing the application database:

```
docker compose exec db createdb -U spvs spvs_test
bash scripts/backend.sh pytest
npm test --prefix dashboard -- --run
npm test --prefix mobile -- --run
npm run build --prefix dashboard
npm run typecheck --prefix mobile
cd mobile
npx expo export --platform android
npx expo export --platform ios
cd ../dashboard
npx playwright install chromium
npx playwright test
```

On macOS the configuration uses Playwright's installed Chromium. Set SPVS_CHROMIUM_PATH only when explicitly selecting another executable. The browser tests read the local demo password in memory, exercise real API endpoints and change only demo data. PostgreSQL tests require a database name ending `_test` and truncate its tables. Unit/contract tests use synthetic sensor and attestation inputs; they are not physical-device verification.

4. Android debug build and run

For emulator-only development set EXPO_PUBLIC_API_URL to http://10.0.2.2:8000/api/v1; for a physical phone set a LAN-reachable server URL. For real data use trusted HTTPS. Set SPVS_DEV_HTTP=1 only for a local HTTP development build.

```
export EXPO_PUBLIC_API_URL=http://10.0.2.2:8000/api/v1
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh android
cd mobile
npx expo start --dev-client
```

The script runs a genuine Gradle assembleDebug and copies its output to release/final/SPVS_ANDROID_DEBUG.apk only on success. Install with `adb install -r android/app/build/outputs/apk/debug/app-debug.apk` from mobile, or use Android Studio. The debug development client needs Metro; it is not a signed production app. Use `adb reverse tcp:8081 tcp:8081` when developing over USB. Configure the API endpoint consistently in the terminal running Metro.

For release distribution remove SPVS_DEV_HTTP, set an HTTPS API URL, configure institution-owned signing keys outside Git and build a reviewed release variant. Never distribute the debug signing key as an institutional credential.

5. iOS project, CocoaPods and simulator

```
cd mobile
gem install bundler
bundle install
cd ..
export EXPO_PUBLIC_API_URL=http://localhost:8000/api/v1
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh ios-simulator
cd mobile
npx expo run:ios
```

The script generates `ios/SPVSStudent.xcworkspace`, installs pods and invokes xcodebuild with simulator SDK and CODE_SIGNING_ALLOWED=NO. The simulator can test navigation/network/permission states; it cannot establish Secure Enclave, App Attest, live-camera liveness or real Wi-Fi behavior. Retain and review the generated Podfile.lock after the first successful native resolution. No validated CocoaPods lock or simulator result is claimed from Linux.

6. Physical iPhone and App Attest

Set an HTTPS API endpoint reachable from the phone, then:

```
unset SPVS_DEV_HTTP
export EXPO_PUBLIC_API_URL=https://YOUR-INSTITUTION-HOST/api/v1
export SPVS_APP_ATTEST_ENV=development
bash scripts/build_mobile_macos.sh ios-device
```

In Xcode select the SPVSStudent target, set a development team, verify bundle ID/capabilities and select a connected iPhone. Enable Developer Mode on the phone and trust the developer profile if requested. Press Run. Start Metro with the same API environment. Camera and location require user consent; select Precise Location. Configure App Attest server trust settings and Apple Developer capabilities exactly as described in CROSS_PLATFORM_IMPLEMENTATION.md. Unsupported or unconfigured App Attest remains unverified. Handle a key-lost error through administrator device reset; never silently replace the registered key.

EAS is optional. The checked-in eas.json defines development, simulator, preview and production profiles. Run `eas init` in your own account, configure EXPO_PUBLIC_API_URL for the selected profile and run `eas build --platform ios --profile development`. Keep signing credentials in EAS or the Keychain. For TestFlight create an App Store Connect app, use production App Attest entitlement/server environment, increment build version, produce a genuine signed archive with Xcode Organizer or EAS production, upload and complete Apple's review/export-compliance steps. No IPA is included unless actually compiled and signed.

7. Demonstration sequence

1. Sign in as administrator, configure a real room, enrollment and the separate Android/iPhone presence policies. Keep iPhone review as the safe default.

2. Sign in as faculty and create a currently active session.

3. On a physical phone sign in as student and register a native device key with password confirmation. Inspect the reported capability in the admin dashboard.

4. Consent to biometric enrollment, permit the camera and follow the randomized center/head-turn/blink prompts. Show the actual result, including retry failures.

5. Select the session and complete fresh identity and location capture. Demonstrate that iPhone current-network data is not a Wi-Fi fingerprint and that review is not attendance approval.

6. Inspect faculty attempts, attendance records, audit events and CSV exports. Test invalid signature/replay using automated tests; do not fake a physical success.

7. Demonstrate face deletion/re-enrollment and an audited lost-device reset. Show pending real-device/provider/calibration work honestly.

8. Troubleshooting

- API unreachable on phone: localhost means the phone itself. Use the Mac LAN IP or trusted HTTPS host, bind API to 0.0.0.0, check firewall and same-network routing. Development HTTP requires a freshly generated development configuration; production remains HTTPS-only.

- Expo Go or missing native module: install a generated development client. Restarting Metro cannot add native modules to an existing binary.

- iPhone Wi-Fi unavailable: verify entitlement, provisioning and Apple's location/privacy requirements. Do not insert dummy BSSID/RSSI values; use review or retry.

- Reduced GPS/poor accuracy: enable Precise Location and retry with a fresh fix. Indoor GPS may remain insufficient.

- Liveness failures: use even light, one face, slow blink and steady capture. Inspect actual timing/orientation on each device. Do not lower mandatory identity rules to force a demo.

- Gradle 403/network/TLS failures: inspect the exact official repository URL. Allow it through the institutional network and configure the JVM proxy/CA correctly. Preserve TLS and artifact verification. Do not mirror an unverified AAR manually.

- Xcode/CocoaPods errors: verify active Xcode, compatible simulator runtime, Ruby/Bundler and generated podspecs. Source code or prebuild success does not prove native compilation.

- App Attest rejected: check TeamID.bundleID, environment, official root fingerprint, matching entitlement and physical-device support. A verified synthetic test certificate is not an Apple attestation.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Deployment guide

Local prerequisites and compatibility

Use Python 3.12, `uv`, Node 24 LTS (or a compatible Node 22 version meeting the locked tool engines), Docker with Compose, and Git. On Apple Silicon, install native ARM64 Python/Node and Docker Desktop for Apple Silicon. PostgreSQL 16 has ARM64 images. OpenCV and MediaPipe wheels must match the interpreter and architecture; avoid mixing x86 Homebrew libraries with ARM64 wheels. This environment's measured platform is Linux x86_64, so macOS native validation remains a manual task.

The normal setup is `make setup`, `make db`, `make migrate`, `make seed`, followed by `make api` and `make dashboard`. The generated `.env` contains unique local secrets. Protect it with restrictive permissions and never include it in an academic submission. For phones, configure `mobile/.env` with a reachable API URL; it contains no credentials.

Linux hosts also need `libgl1`, `libgles2`, `libegl1` and `libglib2.0-0`. The Dockerfile installs these through the distribution package manager. In the restricted cloud workspace, `scripts/cloud_install.sh` extracts verified distribution libraries from a temporary container into ignored local runtime storage; `scripts/backend.sh` activates that location. It does not replace system libraries or disable package/TLS verification. On a normal Debian/Ubuntu machine, installing the packages with the system package manager is simpler.

Docker workflow

1. Generate `.env` and checksum-verified face models.

2. Run `docker compose up --build -d`.

3. Run `docker compose exec backend python -m app.seed` for local demonstration accounts only.

4. Check API `/health` and `/ready`; readiness must load all model assets.

5. Sign into the dashboard and configure real classroom data.

The backend image uses a pinned uv executable and a frozen Python lockfile. The web image uses `npm ci` and nginx. PostgreSQL has a health check and a persistent named volume. API migrations run before uvicorn at container startup. Persistent database files and model files survive process restarts; HTTP processes and authentication sessions in browser memory do not.

Corporate/cloud TLS interception may require the existing trusted CA bundle inside build steps. Both Dockerfiles support an optional BuildKit secret named `ca_bundle`. The cloud installer supplies the host's trusted public CA bundle using `--secret`; it is not baked into the image or included in artifacts. Normal direct-Internet Compose builds need no extra secret. Never use insecure TLS flags as a workaround.

Production-inspired deployment changes

The Compose configuration is a development baseline. Add trusted HTTPS termination, a non-demo administrator provisioning process, institution-specific CORS origins, encrypted backups, key rotation and a retention policy. Remove demo accounts or replace their passwords before real use. Keep the database port private. Use a separate least-privilege application database role and reviewed migration credentials; the development `spvs` role is also the database owner.

Verify `JWT_SECRET` and `BIOMETRIC_KEY` are independent, generated secrets delivered through a secret manager. Rotating the biometric key without re-encryption makes stored templates/device keys

unreadable. Do not invent credentials for provider attestation. Add a provider-backed verifier only with real Google/Apple project credentials and package/signing configuration.

Android development build

Install Android Studio and its supported SDK/NDK tools, enable USB debugging on a test phone and run `npm run android` in `mobile`. The generated project declares the exact SDK/Gradle versions selected by the locked Expo release. `expo prebuild --platform android --no-install` regenerates native code and discovers the local Wi-Fi module. Do not use Expo Go for that module. Gradle compilation, a Metro/Hermes export and physical device tests are distinct checks.

Use a trusted HTTPS URL for distribution. Development HTTP on a local network is only for controlled local testing. Do not put a raw API secret or biometric key in `EXPO_PUBLIC_*`: those values are embedded in the client bundle. iOS builds require Xcode/signing on macOS, and the application reports Wi-Fi scanning as unsupported.

Troubleshooting

Symptom	Diagnosis / action
Database connection failure	Start Compose database, confirm port availability and `.env` URL
Backend config validation fails	Generate real JWT/Fernet secrets; placeholders are intentionally invalid
Readiness returns 503	Run verified model downloader; inspect missing assets and writable caches
Official downloads denied	Allow `media.githubusercontent.com` and `storage.googleapis.com` in environment settings
Model checksum mismatch	Stop using the file; retry official source and investigate upstream provenance
Phone cannot reach API	Use LAN address/HTTPS; `localhost` on a phone points to the phone
Wi-Fi unavailable	Confirm native build, location permission/services and Android scan throttling; follow policy
Frequent liveness retries	Evaluate frame timing, slow blink, pose direction and lighting on the actual device
Browser session disappears on reload	Intended in-memory token storage; sign in again
Duplicate request conflicts	Preserve exact signed payload/key for a retry; use new challenge for new evidence

Do not delete the PostgreSQL volume to resolve ordinary setup issues. `docker compose down -v` is destructive and is not part of normal setup or restart instructions.

Android dependency download in this cloud

The observed Gradle build used Java 17, Gradle 9.4.1, Android API/build tools 37 and NDK 27.1.12297006. React Native 0.88.0’s AAR redirects from Maven Central to `repo.reactnative.dev`; allow that specific domain if the proxy returns HTTP 403. The saved cloud configuration includes this addition, but saving a draft does not apply it. Review/save the environment settings and publish before retrying. Keep TLS verification enabled, using the environment’s trusted Java CA store where necessary. Full native compilation and device execution remain unverified in the delivered results.

Cross-platform release configuration

Apply migrations through `7a2d13ce4f90` before deploying updated clients. Leave ALLOW_LEGACY_DEVICE_REGISTRATION=false; migrate existing HMAC devices through approved resets. Configure ios_presence explicitly, retaining review by default. Set APPLE_APP_ID, APPLE_ATTEST_ENVIRONMENT, APPLE_ATTEST_ROOT_PEM and APPLE_ATTEST_ROOT_SHA256 only after verifying Apple Developer configuration and the official trust anchor; mount the PEM read-only into the backend container. A successful /ready response proves model/database availability, not platform attestation deployment.

Use HTTPS, institution-owned signing credentials and tested backups. Raw frames are transient; encrypted biometric templates persist until explicit deletion. Define and implement institution-approved retention for templates, coarse location summaries, security events and expired challenges before real deployment. Automated retention/cryptographic key rotation and load testing remain operational work; the academic package does not invent a compliance guarantee. Run the physical Android/iPhone matrix and genuine App Attest interoperability checks before enabling production policy. EAS/TestFlight instructions are in SETUP_AND_DEMO_GUIDE.md.

User manual

Administrator

Sign in using the dashboard. In Administration, create staff and student accounts with unique emails and student roll numbers. Assign the faculty account when creating a course, then enroll the correct students. The current role is enforced immediately by the backend; changing a role revokes refresh sessions and device binding and requires an explanation.

In Classrooms, enter surveyed coordinates and a suitable radius. A radius must be 5–300 m. Enter actual Wi-Fi BSSIDs and minimum RSSI values only after collecting a campus reference profile; each line is `aa:bb:cc:dd:ee:ff,-80`. Leave the list empty when unknown, then select a deliberate unavailable policy. The default is manual review. Do not label the demo coordinates as real campus measurements.

Device reset requests appear in Administration. Verify student identity before approving. An approval revokes the previous device; the student must then register the replacement phone. Policy changes are recorded with the old and new values. Requiring attestation while its provider is unconfigured results in review, not silent bypass.

Faculty

Courses lists assigned courses and enrolled students. Sessions opens a bounded attendance window for one course and classroom. Choose local start/end times; the API receives timezone-aware ISO timestamps. Session details refresh approximately every 15 seconds and show attempts separately from official records. Close a session to stop further automatic attendance.

Inspect reasons and risk indicators before reviewing an attempt. In Corrections & reviews, choose the session and student, enter PRESENT/ABSENT/EXCUSED and a meaningful reason. A correction is visibly sourced as CORRECTION and retains its previous status. Resolving a dispute records an explanation; it does not itself create attendance. Make a separate correction if justified. Reports downloads actual authorized records as CSV.

Student

Install the development app and sign in. First register the phone using account-password confirmation. Then open Face enrollment, read the retention explanation and grant consent. Follow the center, turn and slow-blink prompts in even lighting. The server rejects zero/multiple faces, weak samples or incomplete action sequences.

For an open session, tap Verify my attendance. The app checks eligibility/device status, captures a fresh sequence, obtains a current location and gathers supported Wi-Fi observations. It signs the payload and displays the server's result. Do not leave during capture. If a network submission fails, retry the same submission to avoid a new attempt. If the challenge expires or evidence fails, start a fresh challenge when permitted.

History shows official records and attempts separately. Request review of a disputed attempt with an explanation. A PENDING_REVIEW result is not confirmed presence. Settings opens OS permission controls and signs out. Enrollment allows password-confirmed template deletion; Device allows a replacement request. No background location collection is implemented.

Result interpretation

State	Meaning	Next action
PRESENT	All configured gates passed and attendance was recorded	Check history
REJECTED	A mandatory condition failed or a duplicate record exists	Inspect reason; request review if disputed
PENDING_REVIEW	Location/signals/attestation are uncertain under policy	Contact faculty; do not assume attendance approval
REQUIRES_RETRY	Fix a recoverable evidence issue	Enable permissions or obtain a better GPS fix, then retry
EXPIRED	Session/challenge is no longer valid	Ask faculty about a new session or review

Cross-platform completion update

On iPhone allow Camera and Precise Location. SPVS may read the current Wi-Fi BSSID when Apple permits it; it cannot scan nearby networks. Attendance can legitimately remain PENDING_REVIEW under the institution's iPhone policy. Administrators configure iPhone presence independently from Android missing-Wi-Fi policy, with an audit reason. The Device capabilities panel distinguishes legacy signing, reported native storage and verified App Attest registration. Register Apple app attestation only after institutional setup. Reinstall/key loss requires the existing reset-review workflow. To re-enroll a face, delete the existing template with password confirmation, consent again, then capture a new live sequence.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Demonstration script

Before the evaluation

Run the automated suites and retain their current result files. Configure a real classroom and a deliberate Wi-Fi fallback. Complete the real-device checklist with consenting participants before promising a successful live capture. Prepare the randomly generated demo password locally; never place it in slides or the ZIP.

Eight-minute desktop demonstration

1. Explain the two-layer design and the reason uncertain signals produce review.

2. Sign in as administrator. Show the actual course, enrolled demo student and classroom configuration. Point out that initial metrics are real counts rather than seeded attendance.

3. Open a faculty session for the assigned course and a short future window.

4. Show the session page, separate attempts and official record tables, and automatic refresh.

5. Show the policy's manual-review Wi-Fi default and unconfigured attestation status.

6. Sign in as faculty and demonstrate that administrator navigation and API permissions are unavailable. An automated test also verifies cross-course denial.

7. Explain an authorized correction, enter a reason only for a legitimately reviewed record, and show its source/history. Do not misrepresent the correction as biometric verification.

8. Close the session and export the actual CSV. Open the security event log and identify session/correction/export events.

Phone demonstration after hardware validation

Sign in as a student, register the phone, consent to face enrollment and follow the randomized prompts. Open the active session, capture a fresh sequence, collect real location and available Wi-Fi, and show the exact server result. A review/retry result is a valid demonstration of the decision policy; never switch to fake evidence to force PRESENT.

Demonstrate one rejected or reviewable condition: permission denial, a clearly outside location, a revoked device, or a static-photo liveness failure. Then show the corresponding attempt, reason and dispute workflow. Any successful person-to-session result must come from actual captured evidence or be visibly identified as a faculty correction.

If hardware or network is unavailable

Demonstrate the running web/API/database workflow, real-model smoke tests and signed-request protocol tests. State that the API happy-path tests isolate vision and that physical sensor/liveness performance has not been measured. Show `SPVS_VERIFICATION_REPORT.md`, not an invented success video.

Cross-platform completion update

For the cross-platform extension, show Android and iPhone capability differences before a phone demonstration. Open Administration and show the distinct iPhone review/retry/GPS-only choices and required audit reason. Show native device key algorithm and label storage as reported. If no Apple-configured physical device is available, show the explicit unconfigured/unsupported state and synthetic test evidence; do not claim an App Attest demo. Only mark real enrollment and attendance complete after a physical camera run succeeds. Present the final test matrix and outstanding hardware gates alongside the APK, if built.

Final test and build results

Suite	Passed	Failures/errors	Skipped
Backend	109	0	0
Mobile	28	0	0
Dashboard	8	0	0
Browser	3	0	0

Total: **148 passed**. See FINAL_COMPLETION_REPORT.md for test boundaries, commands and intermediate failures. Baseline evidence is retained separately.

Capability	Observed status
Android debug APK	PASS: genuine Gradle assembleDebug, arm64-v8a; debug signed, development client; physical-device testing NOT RUN
Android/iOS JS export	PASS: separate platform exports; final combined export in evidence
iOS native project	PASS: Expo prebuild, camera/location/privacy settings, local podspec/autolinking and static configuration checks
iOS native compilation / simulator / IPA	NOT RUN: Linux host; no macOS/Xcode or Apple signing identity
PostgreSQL migrations	PASS: fresh upgrade, check, full downgrade, re-upgrade, check on isolated PostgreSQL 16 database
Backend and dashboard containers	PASS: both images built and used for real browser/API tests
Physical Android / physical iPhone	NOT TESTED
Apple App Attest	Synthetic verifier/counter tests passed; genuine provider/device verification UNVERIFIED
Google Play Integrity	NOT CONFIGURED; fail-closed boundary only
Biometric calibration	NOT PERFORMED; six real-model smoke cases are not accuracy metrics

No physical Android/iPhone, iOS simulator, Xcode or genuine provider-attestation result is claimed. Synthetic API/sensor/certificate inputs and mocked React Native components are identified in their test modules.